Starter Idea : (A Connectome for Apps)

You stay signed into your usual apps. A helper in the browser can move a task from one to another — but only after you say yes.

Example: you have a client open in your CRM. You ask the helper to make an invoice. It reads the client, shows you the exact details it would send, and only then creates a draft invoice in your invoicing app. The two apps never talk to each other. You are the one who connects them.

That first version only works if both apps are already open, and only for that one job. Later it could grow into a way any opted-in app can offer services to the others, still with you in the loop.

Connectome master reference

A user-authorized Chrome extension reads a client from an opted-in CRM tab and, after the user approves the exact payload, creates an invoice in an opted-in invoicing tab.

That is the whole v0 product. Everything else in this file is context, later work, or a related stack.

The larger idea v0 is meant to start proving is an **app connectome**: opted-in apps expose structured tools; a user-authorized agent in the browser is the only thing allowed to move data and actions between them. Apps do not scrape each other. Apps do not need a proprietary SDK to join. Apps do not know other tabs exist.

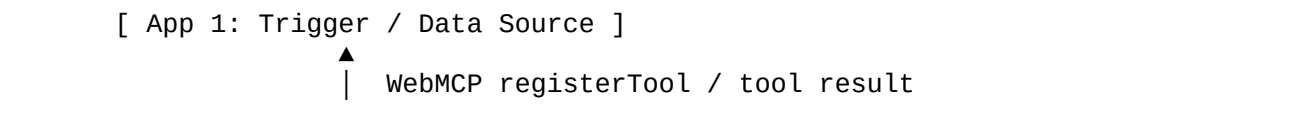
This file is the spec. Implement v0 from §8. Adjacent stacks (personal agents, in-product Vendo) are inventory for later; they are not the hub.

---

1. Architecture

1.1 Hub-and-spoke

Participating web apps stay isolated. They do not do application-to-application (peer-to-peer) communication. A privileged process the user authorizes — a **browser agent / extension**, not the page — is the hub: context router, optional schema translator, and execution engine.



```
                ▼
        [ HUB: browser extension ]
                │  privileged: host_permissions + content scripts
                │  reasoner is behind an interface, not the hub
                ▼
    [ App 2..N: Downstream Targets ]
```

Apps do not know about other active tabs.

Model sentence (never “bypass SOP”):

> Apps stay origin-isolated. A user-authorized browser agent (extension or native) is the only process that may discover tools across tabs and carry data between them.

Direction is valid. Hub-and-spoke with a user-mediated privileged hub is the right shape. Two toy apps can prove it. A production mesh is not in scope until that proof is green.

1.2 Three layers (do not collapse them)

Do not bind the hub to “Hermes Agent Core” or invent `chrome.aiAgent`. Those mix three layers. Keep them separate:

Layer	What it is	Role	v0 choice
WebMCP	Page-level tool contract (`document.modelContext`)	How an app opts in	Real Chrome WebMCP (flag) or same-API polyfill
Browser agent / extension	Privileged hub	Discovers and invokes tools across tabs; holds confirmation	Unpacked Chrome extension. Agent-agnostic.
Reasoner	Plans the chain, maps schemas	Proposes `CRM → invoice` field mapping	Default: 20-line static mapper. LLM optional behind `Mapper`. Hermes is *one* possible reasoner, never the fabric.

Hermes can be one reasoner. The hub must not import Hermes. Otherwise the connectome is a Hermes demo, not an open fabric.

1.3 Discovery → map → execute

The first multi-app chain loop:

```
┌──────────────────┐      ┌──────────────────┐      ┌──────────────────┐
│ 1. Discovery     │ ──→  │ 2. Schema Map    │ ──→  │ 3. Sequential Ex │
│ Open tabs via    │      │ Mapper proposes  │      │ Runs tool calls  │
│ the extension    │      │ target-tool args │      │ tab-by-tab       │
└──────────────────┘      └──────────────────┘      └──────────────────┘
```

1. **Discovery & registration** — compile available WebMCP endpoints.
2. **Schema mapping** — App 1 output rarely matches App 2 input; a mapper (static first, LLM later) proposes args.
3. **Isolated sequential execution** — run tools one tab at a time. Tabs stay separated.

Scanning open tabs is **not** something WebMCP gives you. It is something the extension does because it is privileged. LLM translation “without hardcoded adapter logic” is a demo, not a product principle (see §5.1).

Registry shape — two different things:

Kind	Shape	Lifetime	Use

| Runtime cache (v0) | `{ Tab ID → [tools] }` | Dies when the tab closes | What v0 actually uses |
| Connectome graph (v1+) | `App → Capabilities → Launch / Auth / Risk` | Persisted after the user opts the app in once | The product; not v0 |

A live tab matrix is not a connectome. Tabs are a runtime cache. The graph is the product; mediation is the proof.

1.4 Presence of the target app

Chrome today: visit the site or you cannot see its tools. Clients and browsers must visit a site to know it has tools.

The WebMCP service-worker explainer is the *proposed* answer for “call an app that is not open,” and even that document says multi-origin tool use may not be implementable safely. Background / service-worker tools are **not shipped**.

Phase	If the target tab is missing
v0	Fail with “open the invoicing app.” Do not invent presence.
v1	Open-or-focus the target origin if the user consents to launch it.

Cross-tab `document.modelContext.executeTool` from another document is **spec issue #227**, not a shipped capability. `getTools()` / `executeTool()` only see documents in the **same tab frame tree**. Cross-tab execution is something a **privileged extension** can do (`host_permissions` + content scripts), by invoking `execute` **inside each tab’s page world**.

1.5 How the hub talks to tabs (v0)

Do not invent `chrome.aiAgent`. Do not call `document.modelContext.executeTool` from the extension page and expect it to reach another tab.

Per tab:

1. Content script checks for `document.modelContext`.
2. Discovery: `document.modelContext.getTools()` in that document, or observe `toolchange`.
3. Invoke: in that same document, find the tool and call `document.modelContext.executeTool(tool, args)` **or** call a thin page-side wrapper the stub exposes for the prototype.

The extension background / service worker only routes `{ tabId, toolName, args }` and holds the pending confirmation. It never runs app logic.

Chrome: enable `chrome://flags/#enable-webmcp-testing`. If the native API is missing in the build, a same-API polyfill (MCP-B or a 30-line stub on `document.modelContext`) is allowed so the apps don’t change.

---

2. WebMCP — protocol facts

2.1 Status (as of 17 August 2026)

- **Not** a W3C Proposed Standard. **Not** a W3C Standard. **Not** on the standards track.
- It is a **W3C Web Machine Learning Community Group Draft**: https://webmachinelearning.github.io/webmcp
- Chrome origin trial from Chrome 149: `chrome://flags/#enable-webmcp-testing`
- Chrome Model Context Tool Inspector: https://chromewebstore.google.com/detail/model-context-tool-inspec/gbpdfapgefenggkahomfgkhfehlcenpd
- That is enough to prototype.

Do not claim “W3C standard” in README or comments. Say: *WebMCP Community Group Draft; Chrome origin trial / flag.*

Opt-in via WebMCP is the right door: apps declare capabilities; they do not scrape each other; they do not need a proprietary SDK to join. That matches “open to any app that opts in.”

2.2 API surface

Use `document.modelContext.registerTool({ name, description, inputSchema, execute })`.

Some blogs still mention `navigator.modelContext`; Chrome deprecated that path. Track `document.modelContext`.

`registerTool` returns a `Promise`. Pass `{ signal }` to unregister.

Declarative (HTML-first) and imperative (JavaScript-driven) registration both exist. v0 stubs use imperative.

Use this, not the lookalikes:

Do not use	Use
`handler:`	`execute:`
`type: "text/json"`	`type: "text"` (or a plain JSON-serializable return)
`window.chrome.aiAgent.dispatchToolCall(...)`	Does not exist. Invoke inside the tab’s page world (see §1.5).
Sync `registerTool`	Returns a `Promise`; pass `{ signal }` to unregister

2.3 Origin isolation (WebMCP is built *around* SOP, not through it)

- APIs require a secure, origin-keyed document.
- Permissions-Policy `tools` defaults to `self`.
- Cross-origin sharing is explicit: `exposedTo` + `fromOrigins` + `allow="tools"` on iframes.
- `getTools()` / `executeTool()` only see documents in the same tab frame tree.
- Cross-tab execution is open spec issue [#227](https://github.com/webmachinelearning/webmcp/issues/227).
- Chrome: clients and browsers must visit a site to know it has tools.
- Extensions can query and run WebMCP tools via `host_permissions` + content scripts.

If you keep “bypass SOP,” security reviewers will correctly reject the design.

2.4 Annotations and missing contract pieces

- Use `readOnlyHint` / `untrustedContentHint` as appropriate.
- **No `outputSchema` in the current WebMCP draft** (still [issue #9](https://github.com/webmachinelearning/webmcp/issues/9)).
- Tool descriptions and returned JSON are untrusted. Chrome WebMCP tool security guidance exists specifically because prompt injection is real: App 1’s tool description or returned JSON can become instructions that then act on App 2. https://developer.chrome.com/docs/ai/webmcp/secure-tools

2.5 Existing pieces to reuse rather than invent

- Chrome WebMCP + inspector extension
- [MCP-B](https://docs.mcp-b.ai) if a polyfill / tab transport is needed before native cross-tab exists
- Backend MCP (Vendo MCP door, §11) for apps that already have a server agent
- Hermes only as the reasoner, not as the fabric

---

3. Three user jobs — three products

The connectome ambition is:

1. User **movement** across many apps.
2. Use one app’s services **from another’s interface**.
3. Any app that opts in.

v0 is only the middle third, and only in one shape: an agent sitting beside two already-open tabs, copying data between them. Useful, and the right first slice. It is not yet the connectome.

“From another’s interface” is a different product. Three surfaces:

Product	What the user sees	Status
Agent as cockpit	Extension side panel is the place the user starts the job	**v0. Easiest.**
In-app surface	App A embeds a “use connected apps” affordance	v1 item 4. Closest existing analogue: Vendo overlay / slots / MCP door (§11).
Deep-link / handoff	App A navigates the user into App B with a context token	Later. No launch / focus / session-handoff in v0.

Pick one for v0: **agent as cockpit.** The other two are not features of the same app; they are different products.

Movement: without launch, focus, deep-link, or session-handoff, users do not “move across apps”; the agent does. v0 does not move the user. If the invoicing tab is missing, fail clearly.

---

4. Security, consent, identity

4.1 Threat and product boundary

1. The hub is a privileged agent, not a SOP bypass. State that in one sentence.
2. Three user jobs: *move*, *invoke from another UI*, *chain in the agent*. v0 is the third only.
3. Consent is per edge, writes always confirm, tool I/O is untrusted.
4. Registry is app-scoped and persisted (v1); tabs are just a runtime cache (v0).
5. Agent runtime is swappable.

Security-prompt frameworks and IndexedDB pointer schemas are not the next write. They fall out of a correct boundary. v0 security is §4.3.

4.2 Confused deputy

Tools inherit the signed-in session of each tab. That is good. The hub then becomes a **confused deputy**: it can read CRM and write billing because *you* are logged into both.

Consent has to be **per-edge** (`CRM profile → Invoice create`), not a global “allow this extension to run tools.”

The extension is the confused-deputy surface. Apps stay origin-isolated.

4.3 v0 consent (one edge)

`crm.get-open-client → invoicing.create-invoice`

- **Read tools** may run after the user starts the job (after user intent).
- **Write tools** always confirm, with the **exact payload**. The user is the last schema check.
- Dismiss / navigate-away / tab-close cancels. Nothing is written.
- **No implicit skip** of a write. Stop the chain and show what already happened.
- Tool descriptions and tool results are **untrusted text**. The side panel must render them as data, not as instructions the hub “obeys” without the confirm card.
- Extension host permissions limited to the **two stub origins** in v0. Do not request `<all_urls>` for the proof.
- No “remember this mapping” that auto-writes next time.
- Compensating actions / undo only if the app declared them (not in v0).

That is enough security for two local apps. Do not write a permissions framework yet.

4.4 Prompt injection

LLM mapping is fine for a first experiment **if every write is shown to the user as a typed preview**. Silent field coercion (`clientId` → `customer_ref`, currency, timezones, name vs first/last) will happen. Do not treat “no hardcoded adapters” as a product principle.

4.5 Isolation reminders (for reviewers)

- Secure, origin-keyed document
- Permissions-Policy `tools` defaults to `self`
- Cross-origin sharing only via `exposedTo` + `fromOrigins` + `allow="tools"`
- Same-tab frame tree for native `getTools` / `executeTool`
- Cross-tab only via privileged extension, in the page world

---

5. Mapping, memory, faults

5.1 Schema mapping

Do not treat “the LLM translates payloads instantly, no hardcoded adapters” as the contract. That works on a two-app happy path and fails in production: silent coercion, no `outputSchema`, prompt injection. Fine as an experiment if every write is a typed preview.

v0 ships a static mapper behind this interface:

```ts
interface Mapper {
  map(input: {
    sourceTool: string;
    sourcePayload: unknown;
    targetTool: string;
    targetSchema: object;
  }): Promise<{ args: Record<string, unknown>; notes: string[] }>;
}
```

Ship `StaticClientToInvoiceMapper`. An LLM mapper can implement the same interface later. The confirm card always shows `args` from the mapper. v1 item 5 is “swap in an LLM mapper behind `Mapper`.” The hub must not import Hermes.

5.2 State, memory, tokens

Keep bulky payloads out of the model. Token bloat will kill 3-app chains. Pointer + short summary is the right instinct for later:

- Heavy raw tool outputs (massive JSON tables, raw text) saved in storage instead of the LLM context loop.
- Core prompt memory gets a short text summary plus a pointer.
- Example: *“Successfully extracted profile for Client ID X. Stored payload in DB pointer `_ref_091`. Proceeding to task creation step.”*

Do not implement that in v0.

- *Which origin?* Page origin: other apps cannot read it, and the agent should not inject a shared DB into app pages. Extension origin: the right place for the hub’s scratch space.
- Lifetime, encryption, deletion, and the pointer schema are undefined.
- v0 payloads are small.

v0 choice: no IndexedDB. Keep the payload in **extension memory for the session**.

5.3 Faults

Multi-app chains can cascade. Do not paper that over with “retry twice, mark `DEGRADED`, skip the step, divert around the failed app.” That is retries plus skip, not a circuit breaker. Skip-as-default is unsafe on anything that already mutated state (CRM write, invoice create, payment). Rollback via “undo tools” assumes every opted-in app exposes compensating actions. Most will not.

v0 rules:

- Reads may auto-run after user intent.
- Writes always confirm, exact payload.
- No implicit skip of a write. Stop and show what already happened.
- Compensating actions only if the app declared them.
- **One write, then stop.** On failure, show what ran. No undo, no rollback, no circuit breakers, no retry storm.

---

6. Rejected approaches

These look plausible and are wrong for this project. Do not put them in the repo.

Rejected	Why	Do this instead
Call WebMCP a “W3C Proposed Standard” or “2026 Browser Spec”	It is a Web ML Community Group Draft; not a standard; not on the standards track	“WebMCP Community Group Draft; Chrome origin trial / flag”
“Bypass SOP”	WebMCP is designed around origin isolation. Reviewers will reject the phrase	Privileged hub carries data; apps never talk to each other
Hermes (or any named runtime) as the hub	Mixes reasoner with privileged transport; makes a Hermes demo, not a fabric	Unpacked Chrome extension; reasoner behind `Mapper`
`window.chrome.aiAgent.dispatchToolCall`	Does not exist	Invoke `execute` inside each tab’s page world
`handler:` / `type: "text/json"` / `navigator.modelContext`	Not the current surface	`execute:`, JSON-serializable or `type: "text"`, `document.modelContext`
Native cross-tab `executeTool`	Spec issue #227 is open; native API is same-tab frame tree	Extension content script in the page world
Live `{tab → tools}` as the product	Dies when the tab closes	v0: tab cache. Connectome: persisted `App → Capabilities → Launch / Auth / Risk`
LLM translates instantly, no adapters	Silent coercion, no `outputSchema`, prompt injection	Static mapper; confirm exact JSON; LLM later behind `Mapper`

Retry twice then skip (`DEGRADED`)	Not a circuit breaker; unsafe on writes	Stop. Show what ran.
IndexedDB pointers in v0	Underspecified (origin, lifetime, encryption, schema); payloads are small	Extension memory for the session
Undo tools for rollback	Most apps will not expose compensating actions	None in v0; later only if the app declared them
Background / service-worker tools	Not shipped; multi-origin may not be safely implementable	v0: fail if the tab is missing. v1: open-or-focus with consent
Global “allow this extension to run tools”	Confused deputy across signed-in sessions	Consent per edge
v0 that also moves the user and embeds in App A’s UI	Those are two other products	Cockpit only

---

7. v0 — implement this

7.1 What v0 is for

Prove four things, nothing else:

1. An app can opt in by registering WebMCP tools. It does not know other apps exist.
2. A privileged hub (the extension) is the only process that sees more than one origin.
3. Same-Origin Policy is not bypassed. Apps never talk to each other.
4. A write never runs until the user has seen and approved the exact JSON.

If those hold, the mediation pattern is real. Everything else is v1.

7.2 What v0 is not (leave out on purpose)

Later	Why not now
Persistent capability graph	Tabs are a runtime cache. Graph is the product; mediation is the proof.
Launch / deep-link / “move the user”	If the invoicing tab is missing, fail with “open the invoicing app,” don’t invent presence.
App A UI invoking App B	v0 cockpit is the extension side panel, not an in-app surface.
Hermes, or any named model runtime	Mapping can be a hand-written adapter or a swappable LLM call. The hub must not import Hermes.
Background / service-worker tools	Not shipped. Don’t pretend.
Undo, rollback, circuit breakers	One write, then stop. On failure, show what ran.
IndexedDB pointer store	The payload is small. Keep it in extension memory for the session.
Cross-tab `document.modelContext.executeTool`	Spec issue #227 is open. The extension invokes `execute` **inside each tab’s page world**.

7.3 Roles

Piece	Who	Does
CRM stub	Local web app, origin A	Registers read tools. Owns client records.
Invoicing stub	Local web app, origin B	Registers one write tool. Owns invoices.
Hub	Unpacked Chrome extension	Lists tools per tab, runs the one chain, shows the confirm card.
Reasoner	Optional, behind an interface	Proposes CRM → invoice field mapping. Default is a 20-line static mapper.

7.4 The one user job

Both stubs are already open and signed in (stubs can skip real auth).

1. User opens the extension side panel.
2. Panel shows: CRM tab has `get-open-client`; invoicing tab has `create-invoice`.
3. User clicks **Create invoice from open client** (or types that intent).
4. Hub calls `get-open-client` in the CRM tab.
5. Hub maps the result to `create-invoice` input (static mapper first).
6. Panel shows the **exact JSON** that will be sent, plus a one-line summary.
7. User clicks **Approve**.
8. Hub calls `create-invoice` in the invoicing tab.
9. Invoicing UI updates visibly. Panel shows the invoice id. Chain ends.

If either tab is missing, either tool is missing, or the user dismisses the card: **stop**. No retry storm, no skip, no write.

7.5 Tool contracts

Use the real WebMCP surface. `document.modelContext.registerTool`, callback named `execute`, `registerTool` is async.

CRM — `get-open-client` (read)

```js
await document.modelContext.registerTool({
  name: "get-open-client",
  description: "Returns the client currently open in the CRM UI.",
  annotations: { readOnlyHint: true },
  inputSchema: { type: "object", properties: {} },
  async execute() {
    return {
      clientId: "c_1042",
      name: "River North Studio",
      email: "ap@rivernorth.example",
      billableRate: 180,
      currency: "USD"
    };
  }
});
```

Invoicing — `create-invoice` (write)

```js
await document.modelContext.registerTool({
  name: "create-invoice",
  description: "Creates a draft invoice for a customer. Does not send or charge.",
  inputSchema: {
    type: "object",
    properties: {
      customerName: { type: "string" },
      customerEmail: { type: "string" },
      amount: { type: "number" },
      currency: { type: "string" },
      memo: { type: "string" }
    },
    required: ["customerName", "amount", "currency"]
  },
  async execute({ customerName, customerEmail, amount, currency, memo }) {
    const invoice = appendDraftInvoice({ customerName, customerEmail, amount, currency, memo });
```

```
    return { invoiceId: invoice.id, status: "draft" };
  }
});
```

Optional third tool, still read-only: CRM `list-clients` — only if you want to pick a client other than the open one. Not required for done.

Rules for both apps:

- Tool names: ASCII, hyphens, ≤30 characters.
- Descriptions: what a user would ask for, not the endpoint.
- Writes create a **draft**. No send, no charge.
- Return small JSON. No tables, no HTML, no tool-description text inside the payload.
- Mark any user-sourced field with `untrustedContentHint` if you later add free text from the CRM.

7.6 Done when

A stranger can:

1. Load two stubs and the unpacked extension.
2. Open both apps, open a client in the CRM.
3. Approve one card.
4. See a new **draft** invoice in the invoicing UI, with the CRM client’s name and rate.
5. Repeat with the invoicing tab closed → see a clear stop, no write.
6. Repeat and dismiss the card → no invoice.

No other demo path is required.

7.7 Explicit non-goals for the repo

- No Hermes import, config, or “Nous variant” in the hub.
- No SOP-bypass language in README or comments.
- No IndexedDB, no undo tools, no retry loop.
- No third app.
- No claim of a W3C standard. Say: *WebMCP Community Group Draft; Chrome origin trial / flag.*

7.8 Two-week proof

A 2-week proof is feasible. A production mesh is not.

Do this, and only this:

1. Two local apps (CRM stub, invoicing stub).
2. Each registers 2–3 WebMCP tools with `readOnlyHint` / `untrustedContentHint` as appropriate.
3. Chrome flag + inspector extension, **or** a tiny extension that lists tools per tab and invokes `execute` in that tab’s world.
4. One scripted user intent: “Create an invoice for the client open in the CRM tab.”
5. The mapper proposes a mapping; **the user approves the exact JSON** before the write. Default mapper is static; same confirm card.
6. If the invoicing tab is closed: **stop**, fail clearly, no write.

That proves: opt-in, discovery, mediation, confirmation, no SOP hole.

It does **not** prove: a graph, offline apps, in-app embedding, rollback, or an open ecosystem.

---

8. v1 only after v0 is green

In this order, and only after the six “done when” checks pass:

1. Persist an app-level registry (origin → last-seen tools), still requiring an open tab to execute.
2. Open-or-focus the target origin if the user consents to launch it.
3. Per-edge memory of approved mappings, still confirming each write.
4. In-app “use a connected app” affordance (the other product).
5. Swap in an LLM mapper behind `Mapper`.

Still later / not scheduled in that list: persistent capability graph as a product, deep-link / session-handoff, background service-worker tools, undo/rollback, IndexedDB pointer store, third app, Hermes-in-hub, permissions framework, “remember and auto-write.”

---

9. Decision log

Topic	Options	Decision
Is hub-and-spoke right?	P2P app sockets vs privileged mediator	Privileged mediator; apps ignorant of other tabs
SOP	“Bypass SOP” vs origin isolation + privileged hub	Origin isolation. Never say bypass.
What is v0?	Full connectome vs two-tab mediation	Two-tab mediation in an extension cockpit
v0 user job	Move / in-app invoke / agent chain	Agent chain only
Hub process	Native `chrome.aiAgent` / Hermes core / extension	Unpacked Chrome extension
Reasoner	Hermes locked in vs swappable `Mapper`	Swappable. Default static mapper.
Schema mapping	LLM-only, no adapters vs typed preview + static first	Static mapper; confirm exact JSON; LLM later behind interface
Registry	Live `{tab → tools}` vs persisted `app → capabilities`	v0: tab cache. v1: persist origin → last-seen tools
Missing target app	Pretend background exec vs open/focus vs fail	v0: fail “open the invoicing app”
Cross-tab execute	Native WebMCP vs in-page-world from extension	Extension invokes inside each tab’s page world
Writes	Auto / skip-on-fail / always confirm	Always confirm exact JSON
Reads	Always confirm vs after user intent	After user starts the job
Skip failed write	Graceful skip vs stop	Stop. Show what ran.
Retries	2 retries then DEGRADED vs none	No retry storm
Rollback	Undo tools via IndexedDB vs none	None in v0. Compensating actions only if declared.
Memory	IndexedDB pointers vs session memory	Extension memory for the session
IndexedDB origin if later	Page origin vs extension origin	Extension origin (scratch space)
Consent	Global “allow this agent” vs per-edge	Per-edge
Tool I/O	Trusted vs untrusted	Untrusted text; render as data
Host permissions	`<all_urls>` vs two stub origins	Two stub origins
WebMCP status language	“W3C Proposed Standard” vs CG Draft + origin trial	CG Draft + Chrome flag
API names	`handler`, `text/json`, `navigator.modelContext`	`execute`, JSON-serializable / `text`, `document.modelContext`
Write side effects	Send/charge vs draft	Draft only
Third app	Yes vs no	No
In-app surface	v0 vs v1	v1 item 4
Remember mapping	Auto-write next time vs still confirm	Still confirm each write (v1 item 3)

Polyfill	Wait for native vs MCP-B / 30-line stub	Allowed if native API missing
Personal agents	Grok Bot vs Hermes vs both	Complementary: Grok for app-heavy, Hermes for sovereign/cheap/24-7, Buzz as shared channel
In-product agent	Build a mesh vs install Vendo in the host	Vendo for “agent inside the company’s product”; MCP door for outside agents
Host already has an agent loop	Always full Vendo vs ask	Ask: full Vendo agent vs Vendo tool pack on existing loop
Vendo surface	Overlay vs own chat vs MCP	Question for the human; overlay is recommended for most apps
Vendo model key	Vendo Cloud vs BYO Anthropic/OpenAI	Ask; Cloud is one click; never invent keys
Definition of Vendo-install done	Looks wired vs `vendo doctor --json` exit 0 **and** something visible	Doctor green + visible surface + one concrete first ask

---

10. Grok Bot + Hermes + Buzz (personal agent stack)

This is **not** the connectome hub. It is how two personal agent runtimes complement each other, and how they report into one place. It can sit *behind* the `Mapper` interface later, or act as an **outside agent** through a Vendo MCP door. It must not be imported into the v0 extension hub.

10.1 Complementary, not competitors

	Grok Bot	Hermes
Strength	Cloud computer per agent; sign-ins into any app; model flexibility (Opus, Claude, GPT-5.6); plug-and-play	Full data sovereignty; self-hosted memory you own; low cost floor
Cost shape	~$120–300/month	~$5–10/month
Use for	Deep app integration: Salesforce, Slack, Gmail, Figma, CRMs; working across the whole stack	Sensitive data, cost-heavy volume, full control of memory, anything 24/7 where premium pricing hurts

- **Grok Bot** — anything that needs deep app integration (Salesforce, Slack, Gmail, Figma, CRMs). Built for signing into tools and working across the stack.
- **Hermes** — anything sensitive, cost-heavy, or where you want full control of memory. Financial data, personal admin, anything running 24/7.

Hermes here is the self-hosted Nous-style runtime (MCP, browser automation / Browser Use / CDP, community side-panel). It is not a native WebMCP orchestrator and it does not own `chrome.aiAgent`.

10.2 How to connect them

Primary: Buzz (Jack Dorsey’s open-source team chat). Add Grok Bot agents and Hermes agents to the same workspace; they coordinate in shared channels instead of silos.

Alternative: Grok Bot Chief of Staff delegates specific tasks to Hermes via a webhook or shared task file. Grok is the app-heavy orchestration layer; Hermes is the cheap, always-on execution layer underneath.

10.3 Example setup

- Grok Bot handles the GTM stack: meeting prep, inbox triage, slide generation, CRM updates.
- Hermes runs personal financial research, subscription audits, and content research in the background, cheaper per task.
- Both report into the same Buzz workspace, so you check one place instead of switching between two apps.

10.4 Relation to Connectome v0

Do	Don’t
Treat this as a candidate *reasoner* / personal-ops layer for later	Put Hermes or Grok Bot inside the v0 hub
Use Buzz as *human* coordination between personal agents	Use Buzz as the cross-app tool bus
Let Grok Bot be “app-heavy orchestration” in the *personal* sense (signed-in SaaS)	Confuse that with WebMCP cross-tab mediation

---

11. Vendo — agent inside a host product (and the MCP door)

Vendo puts an agent **inside a company’s product**: customers build views, act through the host’s own API **as themselves**, and automate work inside the host’s brand and guardrails.

It installs as one npm package plus a small `.vendo/` contract in the host repo. A coding agent can complete the whole install; `vendo init` does everything deterministic and tells you exactly what is left.

This is the closest existing stack to:

- **In-app surface** (connectome product 2 / v1 item 4): overlay + slots on the host’s own pages.
- **Backend MCP** for apps that already have a server agent.
- **Outside agents** (Claude, ChatGPT, Cursor, and later a connectome hub) acting in the host through the host’s own guard.

Canonical install page: https://docs.vendo.run/agents.md
Docs index (only if you need more than this + one path page): https://docs.vendo.run/llms.txt

When installing: this section plus **at most one more** page is everything needed. Do step 1, commit to one path, then fetch only that path’s page. Do **not** fetch `llms.txt`, `llms-full.txt`, the existing-agents overview, or a path you are not taking.

Read the repo’s `package.json` before deep directory scans. Start acting after step 1.

Asked for the MCP door? That is outside agents acting in the host’s product through its own guard, riding the `createVendo` composition that both step-1 paths write. It needs the same wiring and visible surface as any other install: do the install, then §11.3.

11.1 Two install paths (branch at step 1)

If the host repo already runs its own agent loop — an `ai` (AI SDK) dependency driving a chat route or `streamText` loop, or `@mastra/core` — **stop and ask** which path:

Path	When	What happens
Full Vendo agent	Human wants Vendo’s agent	Steps 2–10 in order
Tool pack	Human keeps their loop	Fetch **exactly one** walkthrough, follow it end to end, then come back for steps 6 and 7

Walkthroughs (fetch one, not both, not the overview):

- AI SDK loop → https://docs.vendo.run/existing-agents/ai-sdk.md

- Mastra loop → https://docs.vendo.run/existing-agents/mastra.md

The walkthrough wires the human’s chat. It never asks where users keep the screens they build; that question lives in the full-install steps.

Never assume the full install in a repo that already has an agent.

11.2 Full-install flow (steps 2–10)

2. Detect the stack. Read `package.json`.

Dependency	Path
`next`	Next.js
`express`	Express
Anything else (Cloudflare Workers, Bun, Deno, Hono, Fastify, Lambda, bare Node)	`--framework custom` — init writes a runtime-neutral `vendo/server.ts` exporting `handleVendoRequest(request, env)`, mounted in one line

Workspace repos: pick the **app package** first (`apps/web`, `packages/app`, …). A root `package.json` with `workspaces` (or `pnpm-workspace.yaml`) means the framework lives in an app package. Run at the workspace root and detection finds no framework: init errors non-interactively, or lands on the custom scaffold a Next/Express app should not get. Every later command runs in (or is pointed at) that app directory.

3. Install the package, then verify what actually resolved.

```bash
npm install vendoai @vendoai/vendo
npx vendo --version   # must be >= 0.4.0
```

Workspace: install into the app package with the host’s own package manager.

```bash
npm install vendoai @vendoai/vendo --workspace apps/web
yarn workspace <app-package-name> add vendoai @vendoai/vendo
pnpm --filter <app-package-name> add vendoai @vendoai/vendo
```

Then run the version check — and every `npx vendo …` step — from that same app directory.

`vendoai` is a thin alias of `@vendoai/vendo`. Every file init writes imports `@vendoai/vendo/*`. Install **both**: with the alias alone, pnpm’s strict `node_modules` cannot resolve those imports and every route 500s with `Module not found: Can't resolve '@vendoai/vendo/server'`.

The version check is not optional. Stale placeholder versions (0.1.x) exist on npm. Package managers with a release-age cooldown (pnpm 11 `minimumReleaseAge` holds new releases a day by default; npm `min-release-age` is opt-in) will **silently** resolve a fresh Vendo release to that stale version — or to nothing. Everything this install uses (`init --agent`, `vendo login`, `doctor --json`) does not exist there.

If the version is old or missing, exclude Vendo from the cooldown and reinstall:

```yaml
# pnpm-workspace.yaml
minimumReleaseAgeExclude:
  - vendoai
  - "@vendoai/*"
```

npm: unset `min-release-age` for this install, or pin the exact latest version. Re-run the version check before continuing.

Upgrading: if the repo carries a direct `@vendoai/vendo` dependency next to the `vendoai` alias (common in 0.4.1-era installs), bump **both** to the same version. Bumping `vendoai` alone half-upgrades: new CLI, old runtime. `vendo doctor` (0.4.3+) flags the mismatch.

4. Run init in agent mode and relay its questions.

Both commands run from the app package directory. `vendo init` can be pointed at it from the root (`npx vendo init apps/web`), but `vendo login` writes the minted key to `.env.local` **in the directory it runs from** — a login at a workspace root lands the key where the framework never loads it, and everything afterward silently behaves keyless.

```bash
npx vendo init --agent
```

That run writes nothing and exits 0. It prints **one JSON object** holding the questions init still needs answered, each with its options, its recommended pick, and the flag that answers it. Put all of them to the human in the next message, in your own words, in one turn. Keep init’s order and keep its recommendation first.

Usually there are three questions:

1. **How will people use the agent?** Embedded in their app (most apps, **recommended**), through their own agent loop, or from outside AI apps over MCP.
2. **Should the assistant act as their signed-in user**, on the provider init detected?
3. **Where the model comes from:** a free Vendo Cloud key is one browser click and needs no provider account, or they bring their own Anthropic or OpenAI key.

Say what each option gets their users, not what it configures.

Take the model answer first, before re-running anything. It is the one answer that has no flag: the Vendo Cloud pick is a login you run now, and the key only counts once login has written it to `.env.local`.

If they choose Vendo Cloud, drive the login as a **bounded loop** — never a single blocking call:

```bash
npx vendo login --wait 90
```

Rules for login:

- Run this step **alone** — never batched in parallel.
- The moment the URL and code print, **stop** and put both in the next message; the human cannot see the shell.
- Run it **bare** — never pass `--email`, a positional email, or any identity hint. A guessed hint from git config is an assumption.
- Do not background it. Do not wrap it in a shorter timeout. A 90s bounded call returns on its own; a killed call is why you loop.
- Re-run `vendo login --wait 90` until it reports the key landed in `.env.local`. Each re-run resumes the **same** request — no new code, no duplicate key.
- On success the CLI may suggest re-running `vendo init` — that applies only when the repo is not wired yet. If already initialized, go straight to doctor.

- Credential is delivered straight to the CLI (no email OTP; no key ever transits the transcript). Raw protocol: https://vendo.run/auth.md
- If they bring a provider key, never assume it exists — ask where it is set.

Then re-run init with their answers as flags. Example:

```bash
npx vendo init --agent --use-case embedded --auth clerk --byo
```

`status` tells you which run you got: the questions, or the write. Every answer on the first call skips the ask pass and writes in one go — right only when the human has already told you everything.

Init flags:

Flag	Values / meaning
`--use-case`	`embedded` (recommended), own loop, or `mcp`
`--auth`	`authJs`, `clerk`, `supabase`, `auth0`, `jwt`, or `none`
`--byo`	Human brings their own model key
`--cloud-key <key>`	Existing Vendo Cloud key
`--framework`	`next` | `express` | `custom` — required only when detection fails
`--base-url`	Full public URL; for MCP writes into `.env.example`

A `VENDO_API_KEY` already sitting in `.env.local` needs neither `--byo` nor `--cloud-key`: init merges it, uses it, never asks.

If they bring their own key, they put it in `.env.local` as `ANTHROPIC_API_KEY=...` or `OPENAI_API_KEY=...` and tell you when it is there. It never passes through chat.

CLI reference: https://docs.vendo.run/reference/cli#vendo-init-dir
What init writes: https://docs.vendo.run/reference/vendo-init

5. Hand-wire the gaps the agent tail names. The write run ends with an `Agent tail:` block listing the exact files this run left; typically: wrap the root layout (or client entry) in the printed `<VendoProvider>` lines, pass real host components to the provider, and add the auth line if none was wired. Follow the tail, not memory.

Next.js `serverExternalPackages`. Init adds

`serverExternalPackages: ["@vendoai/apps", "esbuild", "@electric-sql/pglite", "@vendoai/store"]`

to `next.config.*` itself, creating the file if the repo has none. It prints the line to paste when the config is not an object literal (function of `phase`, computed export). `@vendoai/apps` is the entry that matters; an `"esbuild"` entry without it is **inert**. Bundled, that import resolves from the app root where pnpm never hoists esbuild, and every generated screen fails its checks while the app itself looks fine. Doctor fails `E-CFG-004` until it is there. On Next 14 the key is `experimental.serverComponentsExternalPackages`.

6. Offer the overlay, then mount it. `VendoProvider` is provider-only and renders **nothing** by itself. Doctor gates on both halves: `E-WIRE-004` until a layout mounts `<VendoProvider>`, `E-WIRE-006` until something visible renders inside it.

The overlay is a floating assistant button on every page, where users ask for things like “show me my top customers as a chart” and get a working screen built right there. **Ask, then mount:**

```tsx
import { VendoOverlay } from "@vendoai/vendo/react";
// inside the VendoProvider wrap:
<VendoOverlay />
```

- Own-loop path: no overlay to place. Keep their chat, loop, and model; wire that chat to render Vendo's guarded tool cards and generated screens inline.
- MCP path: the outside AI app is the surface.

Either way, confirm something **visible** exists before calling the install done: doctor green alone does not prove the human can see anything.

7. Offer slots. A slot is a spot on one of the host's own pages where a user can keep a screen they built (a chart pinned to a dashboard, still there tomorrow). Without one, everything a user builds lives in that user's own list of views.

Three ways out: name a place, let you read the app and bring back options, or skip. Skipping is fine. If they want options, come back with two or three real pages and a reason each, and let them take one, several, or none.

```tsx
import { VendoSlot } from "@vendoai/vendo/react";

<VendoSlot
  id="home-hero"
  label="Dashboard hero"
  description="main dashboard area, where users keep KPI views"
>
  <YourOriginalCard />
</VendoSlot>
```

Always give a `label`: it is what a person picking a destination reads and what an assistant matches a request against — write it as meaning, not as an id. The description is how the assistant finds the right slot by meaning.

- Slot with children: renders them untouched while empty.
- Slot with none: invitation with prompt chips (this space builds itself).
- Filled: one view per person.
- Own-loop path: pass `onAuthor` so an empty slot opens the host's own chat rather than Vendo's palette.

8. Judgment init leaves you (doctor does not grade these):

- **Tool descriptions.** Write each extracted tool as the task a user would ask for, not the endpoint it calls.
- **Risk grades.** Anything destructive or irreversible gets `confirmEach` in `.vendo/overrides.json` (the file that means "a human decided this"; sync never clobbers it).
- **Product brief.** Replace `.vendo/brief.md` placeholder with what this product does and for whom, read out of the code.
- **Theme slots.** Fill anything `.vendo/theme.json` left unresolved from the app's own styles.

9. Gate on doctor. Doctor reads the repo; no server has to be running.

```bash
npx vendo doctor --json
```

Every failing or warning check carries an `error_code` and a `fix_ref` URL. Fix, re-run, repeat until exit 0.

10. Show it running. Doctor green is the gate, but the human should **see** the install. Start the dev server, hand them the exact URL, and give one concrete first ask (e.g. “ask it to build a small dashboard from your data; a guarded write will render an approve/deny card inline”). Name which surface now renders Vendo output (chat page, overlay, embed).

11.3 MCP door (additive; skip none of the install’s own steps)

Outside agents — Claude, ChatGPT, Cursor — acting in the host’s product through its own guard.

Run step 4 with `--use-case mcp` (interactively: **outside agents over MCP**). Init writes (1) and (2) below — composition with `mcp: true`, a thin catch-all route, and the origin-root discovery route:

```bash
npx vendo init --agent --use-case mcp --auth authJs --base-url
https://app.example.com
```

That path needs **Next.js** and one of the four zero-arg presets: `--auth authJs|clerk|supabase|auth0`. On anything else — Express, `--auth none`, or `--auth jwt` (init cannot guess the signing secret) — it prints why and writes nothing MCP; (1) and (2) are yours to wire.

Once the use case is `mcp`, two more questions:

Question	Flags
Who runs OAuth plumbing?	`--posture broker` — Vendo hosts it at `yourcompany.mcp.vendo.run` (uses their Vendo Cloud account); `--posture local` — the app serves it itself with zero config
Will the host backend call these tools machine-to-machine (nightly job)?	`--service-key` sets up a service key

Answer both as flags on the next run. Item 3 (`VENDO_BASE_URL`) is always yours.

1. `mcp: true` plus the OAuth adapter. Written by `--use-case mcp`; otherwise add it to `createVendo({...})`. The door mints its own principals through a `HostOAuthAdapter`; `createVendo` **throws** at composition without one.

Every named preset carries it, so `--auth authJs|clerk|supabase|auth0` (or `jwt({ secret })`) is the whole answer. `--auth none` is **not** — it writes a demo `principal` and no oauth seam, so the door stays shut until that seam is filled.

An `auth` preset owns the seam: a separately passed `oauth` is **ignored** whenever `auth` is set.

```ts
export const vendo = createVendo({
  auth: authJs(), // supplies principal, actAs, and the door's oauth half
  mcp: true,
});
```

No login yet? No preset applies — there is no session to read, and installing a provider’s SDK is not a login. Fill two seams yourself, never beside `auth` (one preset **or** the per-seam trio; mixing throws):

```ts
export const vendo = createVendo({
```

```
  principal: async () => ({ kind: "user", subject: "demo-user" }),
  oauth: {
    session: async () => ({ subject: "demo-user" }),
    principal: async (subject) => ({ kind: "user", subject }),
  },
  mcp: true,
});
```

Leave `actAs` out. A stub returning empty headers is worse than an absent seam, because a fixed demo principal can never act as anyone else. Doctor cannot see the seam, so nothing about it moves step 9's gate. Price of leaving it out: own route-bound tools answer `not-implemented` over the door, while `vendo_make` and the apps tools work. Every client acts as that one person: a demo or an internal tool, not a multi-user product.

2. The discovery route. Also written by `--use-case mcp`. Discovery documents live at **origin-root** paths, outside `/api/vendo`, so the catch-all never sees them:

```ts
// app/.well-known/[...vendo]/route.ts (src/app when the repo uses it)
import { wellKnownVendoHandler } from "@vendoai/vendo/server";
import { vendo } from "@/vendo/server";

export const { GET, POST } = wellKnownVendoHandler(vendo);
```

The handler answers only the door's own paths and 404s everything else under `/.well-known`. Express and custom hosts mount the same composition at the origin root as well (`app.use("/.well-known", mountVendo())`), registered **after** the app's own well-known routes.

3. `VENDO_BASE_URL`. Set it to the deployment's **full public URL**, path prefix included. Issuer, advertised endpoints, protected-resource identifier, and RFC 8707 audience all derive from it. Left unset they come from the request URL — behind any proxy, the proxy-internal origin, so discovery advertises endpoints no client can reach. Forwarded headers are **never** consulted.

```bash
VENDO_BASE_URL=https://app.example.com
```

`--base-url` only writes it into `.env.example`; setting it where the host deploys stays the human's. Doctor fails `E-MCP-009` on an MCP-wired project with neither this variable nor `mcp: { baseUrl }`.

4. `serviceAuth` — only when asked for. Skip unless the **host's own backend** has to act for a user who is not at a browser (nightly job, queue worker). `mcp: { serviceAuth: { keys: [...] } }` opens an RFC 8693 exchange at the door's own token endpoint: the backend posts a key plus one of the host's user ids and gets a short-lived token bound to that user.

Keys are opaque secrets the human generates — ask before creating one. On the MCP init path, `--service-key` generates one into `.env.local` and wires it, **only under local posture**: a broker-fronted door serves no token endpoint of its own, so the key is the console's to create there.

Third-party agents need none of it; they run the per-user OAuth the door already serves.

5. Verify: step 9's gate, nothing new. Registry checks grade `server.json` and the challenge only when those files already exist — publishing to the MCP registry is a separate job, not part of reaching green.

11.4 Rules of engagement

- **Ask the human before creating any account or key.** Includes Vendo Cloud (`vendo login`), model provider keys, and sandbox accounts (E2B or managed Cloud sandbox). Relay the choice; never sign up on your own. Look before you ask: a `VENDO_API_KEY` already in `.env.local` or the environment is used as it is.
- **Never invent props or tools outside the catalog.** Only components registered in `vendo/registry.tsx` exist, and only with the props their schemas declare. Only tools in `.vendo/tools.json` exist. Copy real names from source.
- **Doctor green is the only definition of done.** `vendo doctor --json` exiting 0 (no `broken` checks) is complete; anything less is not — *plus* step 10: the human must see a visible surface.
- **Run `vendo init` before touching `.vendo/`.** Init writes the whole `.vendo/` contract (`tools.json`, `overrides.json`, `policy.json`, `brief.md`, `theme.json`). Don't author those files before the first init; re-run `npx vendo init` instead of hand-writing scaffold.
- **The star ask acts only on an explicit yes.** Prompt ends by asking whether to star the repo. If yes and `gh` is authenticated: `gh api --method PUT /user/starred/runvendo/vendo`; otherwise print `https://vendo.run/star?src=agent`. If no or unanswered, drop it.

11.5 Surfaces and further pages

Surface	What it serves
https://vendo.run/agents.md	Install instructions as raw Markdown (append `.md` to any docs page)
https://docs.vendo.run/existing-agents.md	Tool-pack path for hosts that already have an agent
https://vendo.run/auth.md	Raw claim-ceremony protocol behind `vendo login`
https://docs.vendo.run/llms.txt	Index of every docs page for LLM ingestion (also `/.well-known/llms.txt`)
https://docs.vendo.run/llms-full.txt	Whole docs site as one file
`npx vendo init --agent`	Init's setup questions as one JSON object, answered back as flags
`npx vendo sync --json`	One machine-readable sync report object on stdout
`.claude/skills/vendo-setup/`	Setup skill shipped in the npm tarball; init writes it when `.claude/` exists and it is missing; never overwrites an edited copy

Further pages:

- Host auth — detect provider, wire preset, know when to ask: `/production/auth`
- API tools — expose host API as tools; what the extractor reads: `/capabilities/api-tools`
- Troubleshooting — every doctor error code: `/production/troubleshooting`
- Edge runtimes: `/production/edge-runtimes`
- Tools and catalog: `/generated/host-components`
- Slots: `/product/mount-the-surface#slots`
- MCP door internals: `/outside-agents/how-the-door-works`
- `E-CFG-004`: `/production/troubleshooting/e-cfg-004`
- `E-MCP-009`: `/production/troubleshooting/e-mcp-009`
- MCP verify codes: `/production/troubleshooting/e-mcp-004`

11.6 Concepts

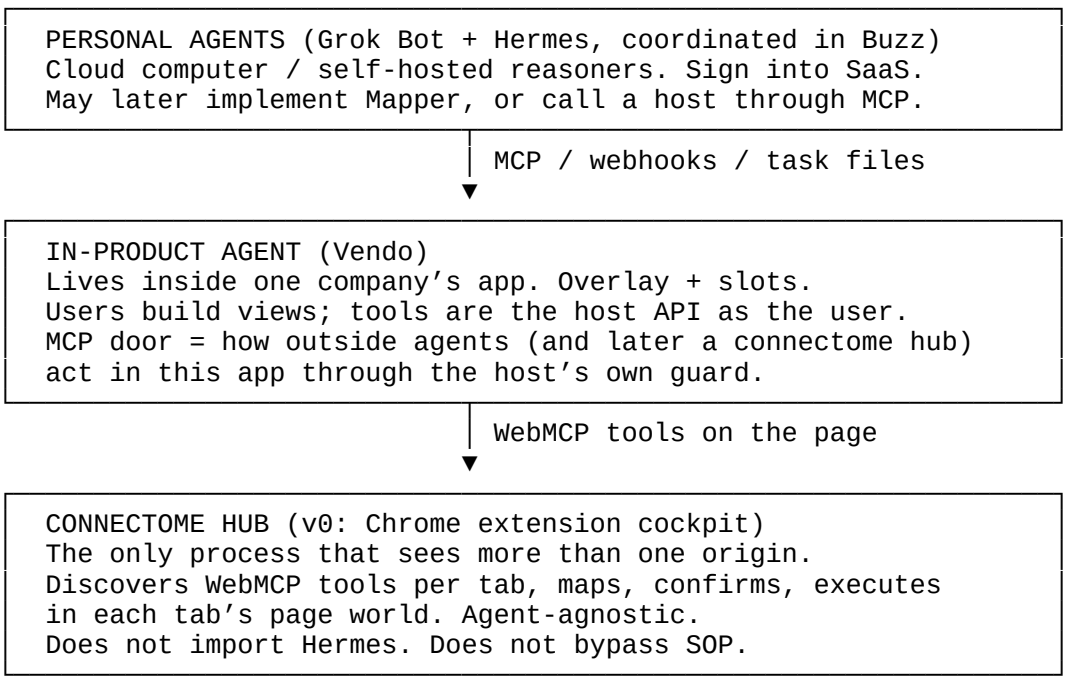
Concept	Meaning
Host	The company's product Vendo is installed into
Act as themselves	Tools run as the signed-in user, through the host API, inside the host's guardrails
`.vendo/` contract	`tools.json`, `overrides.json`, `policy.json`, `brief.md`, `theme.json` — written by init

Overlay	Floating assistant; visible half of the install
Slot	Named place on a host page where a per-user generated view can live
MCP door	Outside agents enter through OAuth + MCP, still guarded by `createVendo`
Broker posture	Vendo hosts OAuth at `yourcompany.mcp.vendo.run`
Local posture	The app serves OAuth itself
`confirmEach`	Human-decided risk grade in `overrides.json`; sync will not clobber
Doctor	Repo-local verifier; exit 0 is the gate

---

12. How the pieces fit together

Three different “agent” ideas. They compose; they are not the same product.



v0 uses only the bottom box, plus two toy apps that register WebMCP tools. Vendo and Grok/Hermes/Buzz are inventory for later:

Later need	Reach for
In-app “use connected apps” (v1 item 4)	Vendo overlay / slots as a pattern; or a thinner affordance that asks the hub
App that already has a server agent	Vendo MCP door (or any backend MCP) as a spoke, not instead of WebMCP on the page
LLM mapper (v1 item 5)	Anything behind `Mapper` — including Hermes or Grok Bot
Personal 24/7 / sensitive work	Hermes, not the connectome hub
Personal GTM / signed-in SaaS orchestration	Grok Bot, not the connectome hub
Human-visible coordination of those personal agents	Buzz
Polyfill before native WebMCP / cross-tab	MCP-B or a 30-line `document.modelContext` stub
Inspect tools while developing	Chrome Model Context Tool Inspector

What v0 must not become: a Hermes demo, a Vendo install, or a Buzz workspace. Those can surround it later.

---

13. Glossary

Term	Meaning here
Connectome	Persistent graph of opted-in apps and capabilities, plus a privileged hub that may move data/actions between them with per-edge consent
Mediation pattern	Hub sits beside open tabs, discovers tools, maps, confirms, executes. v0 is this pattern, not the full connectome
Spoke	An opted-in app exposing WebMCP tools. Ignorant of other spokes
Hub	Privileged browser extension (v0). Not Hermes, not `chrome.aiAgent`
Reasoner / Mapper	Swappable component that proposes target-tool args from source payload
Edge (consent)	One directed pair of tools, e.g. `crm.get-open-client → invoicing.create-invoice`
Confused deputy	Hub inherits both tab sessions, so it can do things neither app would allow the other
Cockpit	Extension side panel as the UX for starting and confirming a chain
WebMCP	Page-level tool contract (`document.modelContext`). CG Draft, Chrome origin trial
MCP	Model Context Protocol (server/agent side). Vendo’s “MCP door” is this, for outside agents
MCP-B	Polyfill / tab transport if native WebMCP or cross-tab is missing
SOP	Same-Origin Policy. Stays intact. Not bypassed
Hermes	Nous self-hosted agent runtime (MCP, browser automation, community extension). Reasoner candidate, not the hub
Grok Bot	Cloud-computer personal agent with app sign-ins and model flexibility
Buzz	Open-source team chat used as the shared channel for personal agents
Vendo	npm package + `.vendo/` contract that puts a guarded agent inside a host product
Overlay	Vendo’s floating in-app assistant button
Slot	Vendo mount point on a host page for a per-user generated view
MCP door	Vendo’s OAuth + MCP entry for outside agents, still under `createVendo` guardrails
Doctor	`vendo doctor --json` — definition of a complete Vendo install (exit 0)

---

14. Open spec / product issues

ID / item	Why it matters
WebMCP [#227](https://github.com/webmachinelearning/webmcp/issues/227)	Cross-tab tool execution is not native. v0 uses the extension page-world workaround.
WebMCP [#9](https://github.com/webmachinelearning/webmcp/issues/9)	No `outputSchema` yet. Mapping cannot be a typed contract at the protocol layer.
[Service-worker explainer](https://github.com/webmachinelearning/webmcp/blob/main/docs/service-workers.md)	Proposed “call an app that is not open.” Not shipped; multi-origin may not be safely implementable. v0 must not pretend.
Chrome WebMCP tool security	Prompt injection via tool descriptions and results. Confirm writes; treat I/O as untrusted.
Presence / launch	Chrome: visit the site or you cannot see tools. v1 item 2 is open-or-focus with consent.
Persistent graph	The actual connectome product. After v0 mediation is proven.

---

15. Links

WebMCP / Chrome

- Spec (CG Draft): https://webmachinelearning.github.io/webmcp
- Chrome flag: `chrome://flags/#enable-webmcp-testing`
- Inspector: https://chromewebstore.google.com/detail/model-context-tool-inspec/gbpdfapgefenggkahomfgkhfehlcenpd
- Secure tools: https://developer.chrome.com/docs/ai/webmcp/secure-tools
- Issue 227 (cross-tab): https://github.com/webmachinelearning/webmcp/issues/227
- Issue 9 (outputSchema): https://github.com/webmachinelearning/webmcp/issues/9
- Service workers explainer: https://github.com/webmachinelearning/webmcp/blob/main/docs/service-workers.md
- MCP-B: https://docs.mcp-b.ai

Vendo

- Install page: https://docs.vendo.run/agents.md
- Auth ceremony: https://vendo.run/auth.md
- Existing agents: https://docs.vendo.run/existing-agents.md
- Docs index: https://docs.vendo.run/llms.txt

---

16. Quick start for the next implementer

1. Implement **§7** only. Two origins, one extension, one confirmed write, six done-when checks.
2. Copy tool contracts from §7.5, hub behavior from §1.5, consent from §4.3, mapper from §5.1.
3. If an older sketch disagrees with §7 or §9, ignore the sketch.
4. Do not install Vendo, Hermes, Grok Bot, Buzz, or IndexedDB to ship v0.
5. After the six checks pass, take v1 in the order in §8.

If you are instead installing Vendo into a host app, ignore §7’s stubs and follow §11 end to end, still asking the human at every account/key/path branch.